USEI11 – Directed Line Upgrade Plan

Contexto

Assume-se que as linhas ferroviárias são direcionadas, representando dependências técnicas entre estações. Uma aresta A → B significa que a estação A deve ser atualizada antes da estação B (por exemplo, devido a sistemas de sinalização, controlo ou compatibilidade técnica). Assim, a rede forma um grafo dirigido, onde é necessário:

- Determinar uma ordem válida de atualização das estações;
- Detetar ciclos de dependência, que tornam o planeamento impossível.

User Story

Como planeador de infraestruturas, quero calcular uma ordem válida de atualização das estações, respeitando as dependências direcionais entre elas, e identificar ciclos, caso existam, para poder corrigir conflitos no planeamento.

Análise da Complexidade Temporal

Variáveis de Análise

Símbolo	Significado
V	Número de vértices (estações)
E	Número de arestas (ligações direcionadas)

Estrutura Geral do Algoritmo

O método topologSort(Graph<V,E> g) é dividido em duas fases:

- Deteção de ciclos (colorDFS)
- Ordenação topológica (topoDFS)

Método Principal

Map<String, Object> topologSort(Graph<V, E> g)

Linha	Código	Complexidade Temporal	Justificação
1	Map<String, Object> result = new HashMap<>();	**O(1)**	Inicialização de HashMap vazio (operação constante)
2	(g == null || g. numVertices() == 0)	**O(1)**	Verificação de condições simples
3	result.put("hasCycle", false);	**O(1)**	Inserção em HashMap
4	result.put("topsort", new LinkedList<V>());	**O(1)**	Criação de lista vazia + inserção em HashMap
5	return result;	**O(1)**	Retorno imediato
6	int[] color = new int[g. numVertices()];	**O(V)**	Inicialização de array de tamanho V
7	Deque<V> stack = new ArrayDeque<>();	**O(1)**	Criação de pilha vazia
8	Set<V> cycleVertices = new LinkedHashSet<>();	**O(1)**	Criação de conjunto vazio
9	for (V v : g.vertices())	**O(V)**	Itera sobre todos os V vértices do grafo
10	if (color[g.key(v)] == 0)	**O(V)**	Chamada a key() dentro do loop
11	if (colorDFS(g, v, color, stack, cycleVertices))	**O(V × (V + E))**	Custo acumulado de todas as

Linha	Código	Complexidade Temporal	Justificação
			chamadas de colorDFS
12	result.put("hasCycle", true);	**O(1)**	Inserção em HashMap
13	result.put("cycleVertices", cycleVertices);	**O(1)**	Inserção em HashMap
14	return result;	**O(1)**	Retorno ao encontrar ciclo
15	boolean[] visited = new boolean[g. numVertices()];	**O(V)**	Inicialização de array de tamanho V
16	LinkedList<V> topsort = new LinkedList<>();	**O(1)**	Criação de lista vazia
17	for (V v : g.vertices())	**O(V)**	Itera sobre todos os V vértices do grafo
18	if (!visited[g.key(v)])	**O(V)**	Chamada a key() dentro do loop
19	topoDFS(g, v, visited, topsort);	**O(V × (V + E))**	Custo acumulado de todas as chamadas de topoDFS
20	result.put("hasCycle", false);	**O(1)**	Inserção em HashMap
21	result.put("topsort", topsort);	**O(1)**	Inserção em HashMap
22	return result;	**O(1)**	Retorno final

Complexidade Temporal Total:

1. Deteção de Ciclos

- Loop externo sobre os vértices - O(V)
- Cada chamada a colorDFS custa - O(V · (V + E))

2. Ordenação Topológica

- Loop externo sobre os vértices - O(V)
- Cada chamada a topoDFS custa - O(V · (V + E))

Soma Total:

.: O(V × (V + E)) + O(V × (V + E)) = **O(V·(V + E))**

Métodos Auxiliares

Deteção de Ciclos – boolean colorDFS(Graph<V, E> g, V vOrig, int[] color, Deque stack, Set cycleVertices)

Descrição

Utiliza uma DFS com marcação por cores:

- WHITE (0) → não visitado
- GRAY (1) → em processamento
- BLACK (2) → completamente processado
- Se durante a DFS for encontrado um vértice GRAY, existe um ciclo.

Linha	Código	Complexidade Temporal	Justificação
1	final int WHITE = 0, GRAY = 1, BLACK = 2;	**O(1)**	Definição de constantes (valores fixos)
2	color[g.key(vOrig)] = GRAY;	**O(V)**	Chamada a key() que percorre lista via indexOf()
3	stack.push(vOrig);	**O(1)**	Inserção no topo da pilha (operação constante)
4	for (V vAdj : g.adjVertices(vOrig))	**O(E)**	Loop sobre vértices adjacentes a vOrig (arestas)
5	int adjKey = g.key(vAdj);	**O(V)**	Chamada a key() dentro do loop
6	if (color[adjKey] == GRAY)	**O(1)**	Comparação simples de inteiros
7	for (V v : stack)	**O(V)**	Percorre a pilha (no pior caso contém V vértices)
8	cycleVertices.add(v);	**O(1)**	Inserção em LinkedHashSet

Linha	Código	Complexidade Temporal	Justificação
9	if (v.equals(vAdj)) break;	**O(1)**	Comparação e possível quebra do loop
10	return true;	**O(1)**	Retorno imediato
11	if (color[adjKey] == WHITE)	**O(1)**	Comparação simples de inteiros
12	if (colorDFS(g, vAdj, color, stack, cycleVertices))	**—**	Chamada recursiva
13	return true;	**O(1)**	Retorno imediato caso ciclo seja encontrado
14	stack.pop();	**O(1)**	Remoção do topo da pilha (operação constante)
15	color[g.key(vOrig)] = BLACK;	**O(V)**	Chamada a key() que percorre lista via indexOf()
16	return false;	**O(1)**	Retorno quando nenhum ciclo é encontrado

---

Complexidade Temporal Total:

1. Operações feitas uma vez por vértice

Operação	Custo
color[g.key(v)] = GRAY	O(V)
stack.push(v)	O(1)
color[g.key(v)] = BLACK	O(V)

- Total por vértice: O(V + 1 + V) = O(V)
- Como existem V vértices, temos: O(V × V) = O(V^2)

2. Percurso das arestas (loop)

Operação	**Custo**
g.key(vAdj)	O(V)
comparação de cor	O(1)
possível chamada recursiva	contabilizada globalmente

- Custo por aresta: O(V)
- Total para todas as arestas: O(E × V)

3. Construção do ciclo (quando encontrado)

Percorre a stack até encontrar o vértice - O(V) Isto acontece no máximo uma vez, porque o algoritmo termina ao encontrar um ciclo.

- Custo adicional: O(V)

Soma Total:

.: $O(V^2 + E \cdot V + V)$ = **$O(V \cdot (V + E))$**

Ordenação Topológica – void topoDFS(Graph<V, E> g, V vOrig, boolean[] visited, LinkedList topsort)

Descrição

Executada apenas se não existir ciclo. Utiliza DFS e adiciona cada vértice à frente da lista após processar todos os seus sucessores.

Linha	Código	Complexidade Temporal	Justificação
1	visited[g.key(vOrig)] = true;	**O(V)**	Chamada a key() que percorre lista via indexOf()
2	for (V vAdj : g.adjVertices(vOrig))	**O(E)**	Loop sobre vértices adjacentes a vOrig (arestas)
3	if (!visited[g.key(vAdj)])	**O(V)**	Chamada a key() dentro do loop
4	topoDFS(g, vAdj, visited, topsort);	**—**	Chamada recursiva (custo contabilizado globalmente)
5	topsort.addFirst(vOrig);	**O(1)**	Inserção no início da LinkedList (operação constante)

Complexidade Temporal Total:

1. Operações feitas uma vez por vértice

Cada vértice executa:

Operação	Custo
visited[g.key(v)]	O(V)
topsort.addFirst(v)	O(1)

- Total por vértice: O(V)
- Com V vértices: $O(V^2)$

2. Percurso das arestas (loop)

Operação	Custo
g.key(vAdj)	O(V)
if (!visited[...])	O(1)

- Custo por aresta: O(V)
- Total para todas as arestas: O(E × V)

Soma Total:

.: $O(V^2 + E \cdot V)$ = **$O(V \cdot (V + E))$**

Considerações Finais

A análise de complexidade temporal do método topologSort mostra que o custo dominante está associado às chamadas repetidas aos métodos colorDFS e topoDFS, ambos dependentes da operação key(), cujo custo é O(V) por chamada. Como consequência:

- A deteção de ciclos tem complexidade $O(V \cdot (V + E))$
- A ordenação topológica, quando executada, também tem complexidade $O(V \cdot (V + E))$
- Assim, a complexidade total do algoritmo é: $O(V \cdot (V + E))$